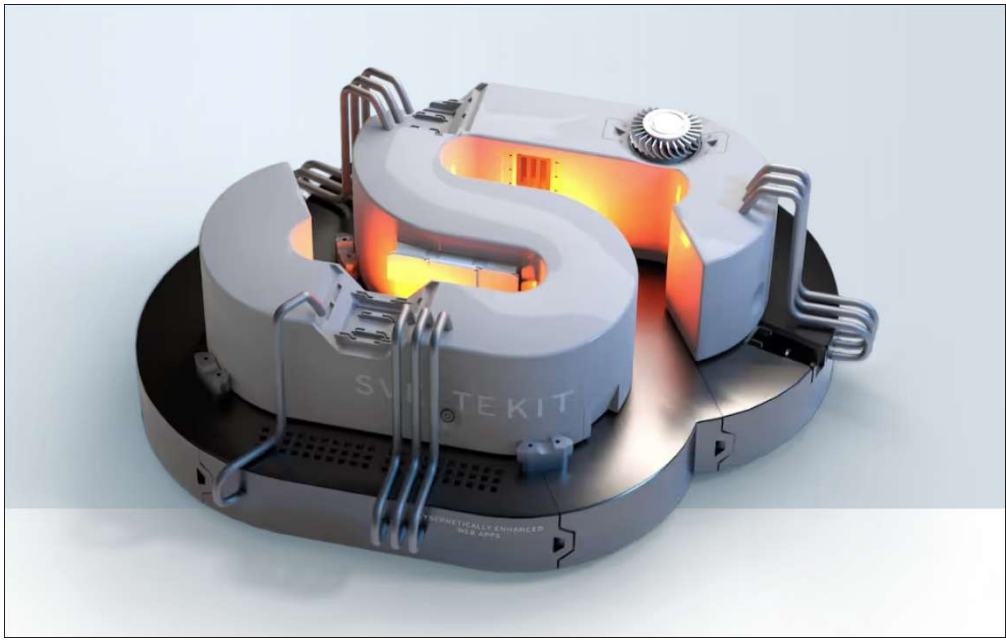
Voor jouw apotheek

Svelte Fundamentals

Module – SvelteKit introduction

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training



SvelteKit

Preferred way for creating complete Svelte Applications instead of ‘just Svelte pages’

What is SvelteKit?

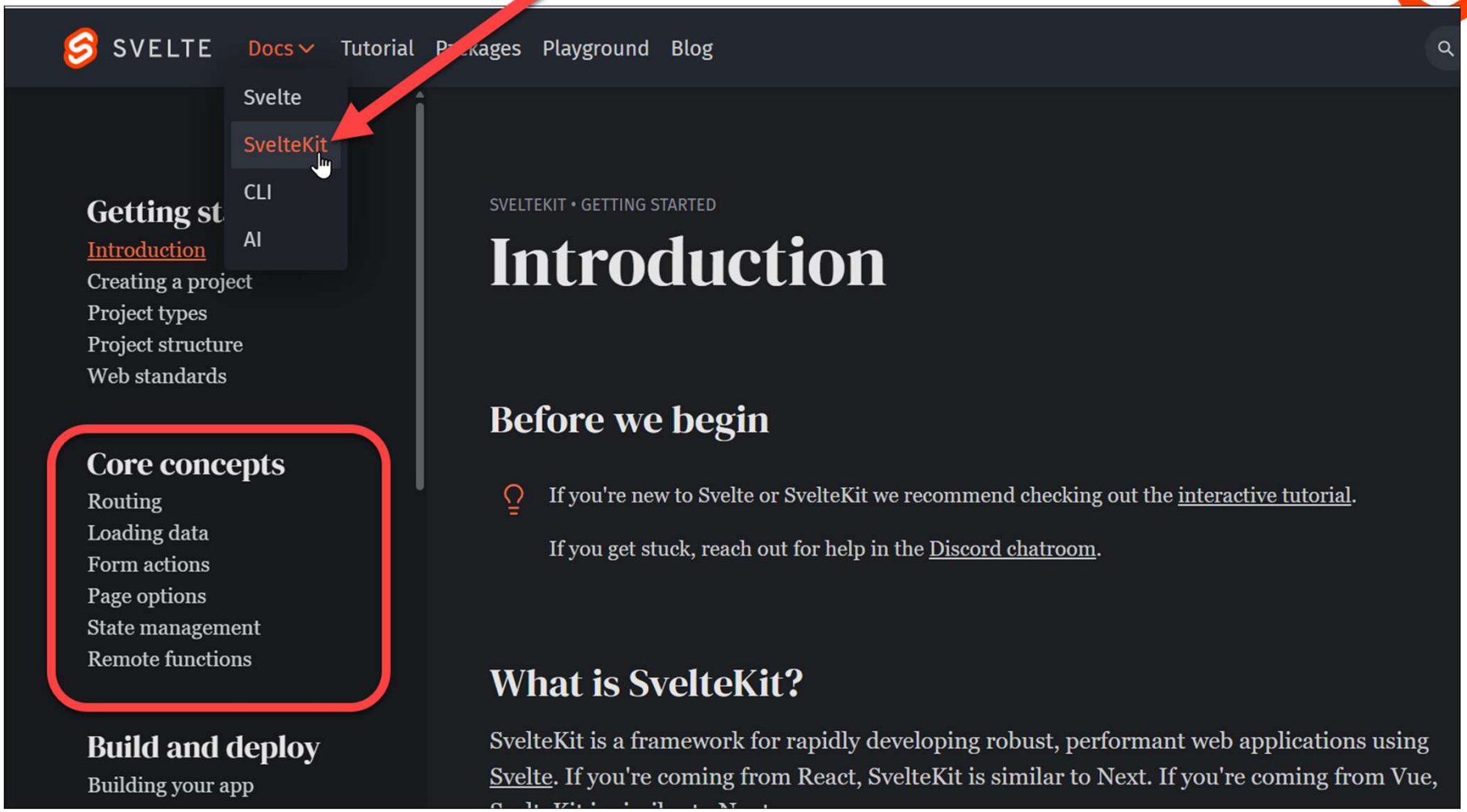
"SvelteKit is a framework built on top of Svelte – so, a framework on top of a framework"

https://svelte.dev/docs/kit/introduction

Goals of SvelteKit

- Svelte renders UI components. SvelteKit helps you *build web apps*.
- Core pillars
  - File based routing system
  - Server-sided data loading
  - Overcoming the client/server boundary (API/Server endpoints)

Svelte documentation



https://svelte.dev/docs/kit/introduction

Creating a project + structure

- Using `npx` command line tool, or Vite

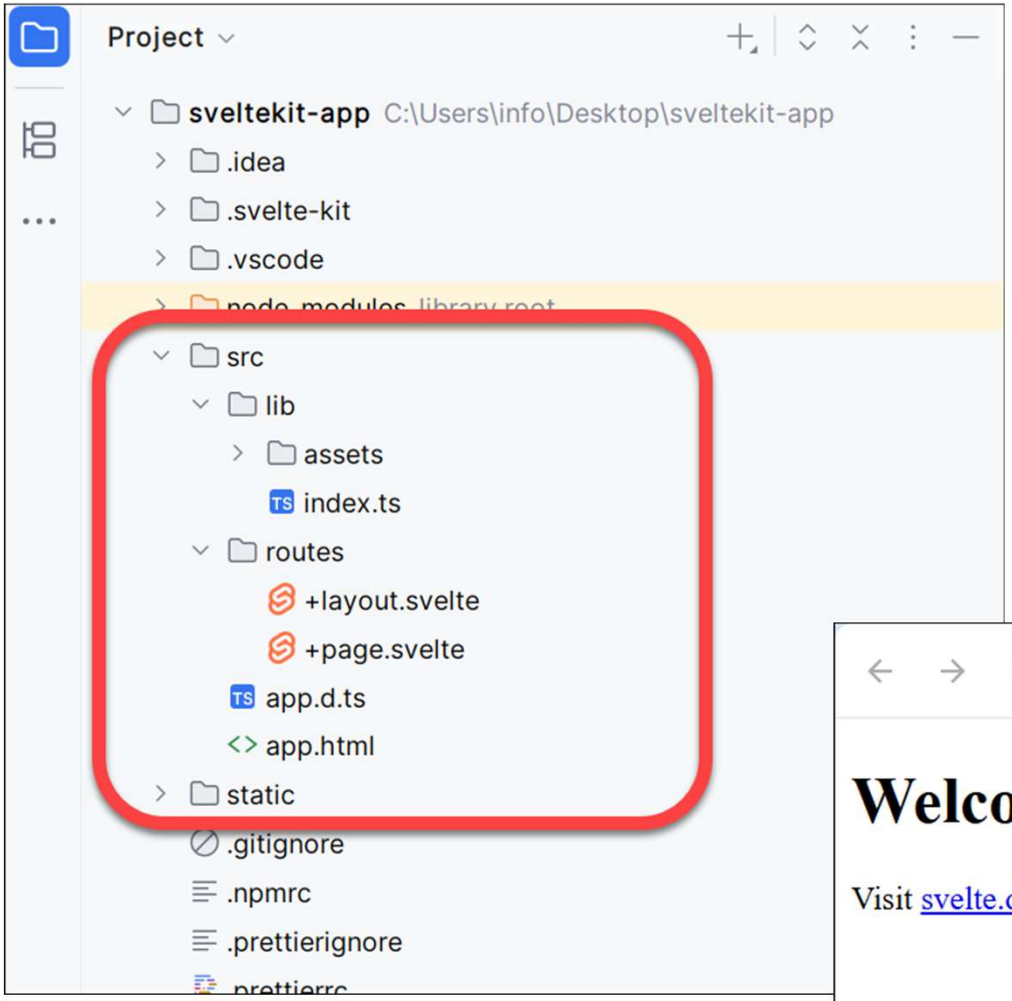
`npx sv create sveltekit-app` (or any name you like)

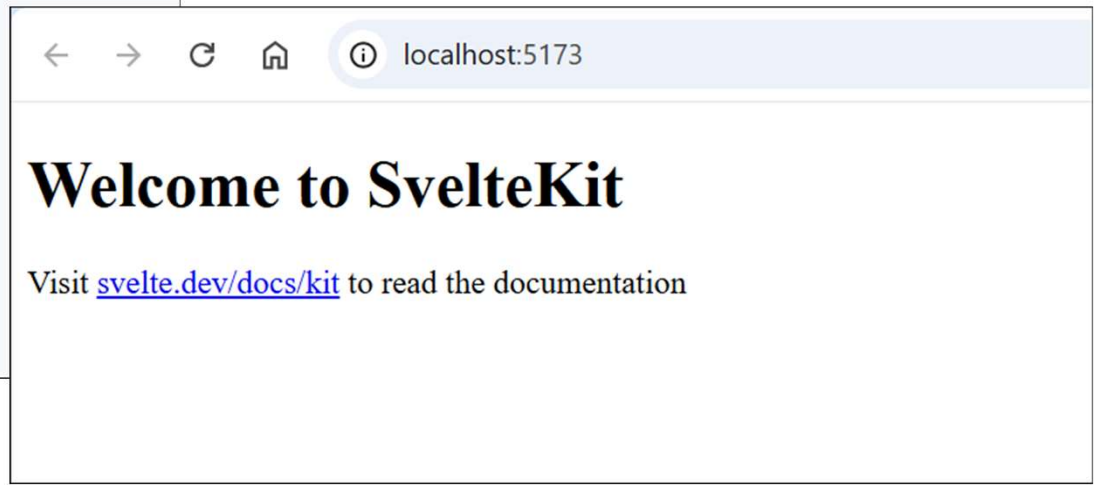
```
PS C:\Users\info\Desktop> npx sv create sveltekit-app
Need to install the following packages:
sv@0.12.7
Ok to proceed? (y)


  Welcome to the Svelte CLI! (v0.12.7)

  Which template would you like?
  ● SvelteKit minimal (barebones scaffolding for your new app)
  ○ SvelteKit demo
  ○ Svelte library
```

Pick your options

```
Welcome to the Svelte CLI! (v0.12.7)

Which template would you like?
SvelteKit minimal

Add type checking with TypeScript?
Yes, using TypeScript syntax

What would you like to add to your project? (use arrow keys / space bar)
prettier

Project created

Successfully setup add-ons: prettier

Which package manager do you want to install dependencies with?
npm

To skip prompts next time, run:
npx sv@0.12.7 create --template minimal --types ts --add prettier --install npm sveltekit-app

Installing dependencies with npm ...
```

Minimal project structure





Important folders

- `src/routes`
  - Every file (which is a Svelte Component) is a page in your app
  - There are special file names with a special meaning (`+page.svelte`, `+layout.svelte` and more)
- `src/lib`
  - Contains the rest of your plain Svelte components
  - Create subfolders where necessary/logical
  - Every page can be composed of multiple components

More options on project structure:

https://svelte.dev/docs/kit/project-structure

Project structure

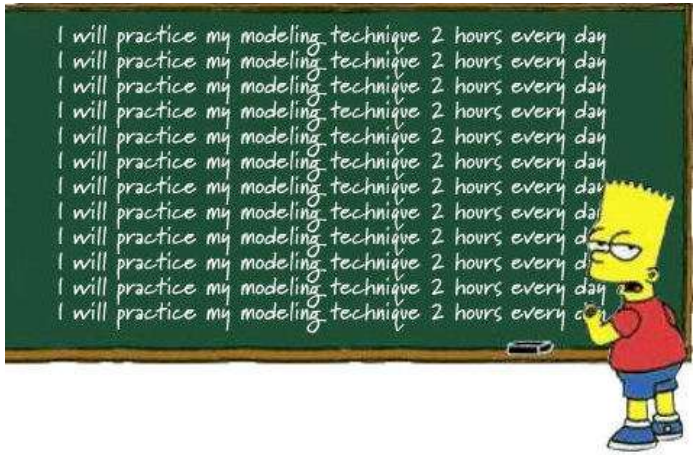
A typical SvelteKit project looks like this:

```
my-project/
├ src/
│ ├ lib/
│ │ ├ server/
│ │ │ └ [your server-only lib files]
│ │ └ [your lib files]
│ ├ params/
│ │ └ [your param matchers]
│ ├ routes/
│ │ └ [your routes]
│ ├ app.html
│ ├ error.html
│ ├ hooks.client.js
│ ├ hooks.server.js
│ ├ service-worker.js
│ └ instrumentation.server.js
├ static/
│ └ [your static assets]
├ tests/
│ └ [your tests]
├ package.json
├ svelte.config.js
├ tsconfig.json
└ vite.config.js
```

Can be *.js or *.ts files

Mini Workshop

- Go ahead and create your first SvelteKit project
  - Use `npx sv create <name>` or `npm create vite` and work from there
- Make your own choices in the cli prompts
- Run the new application
- Create a new component and use it inside `+page.svelte`
- What does `$lib` alias do?
- Explore the configuration file `vite.config.ts`



Sidestep: Bootstrap

Installing external CSS Libraries in your SvelteKit application

Sidestep – installing Bootstrap

- But, this applies in general to external libraries you want to use in Nuxt
  - Bootstrap, Concept7, Material Design, etc.
  - Some libraries have special Nuxt Modules *(we'll cover them later!)*
- Install dependencies in project:
  - `Npm install bootstrap`
  - `Npm install bootstrap-icons`
- Dependencies are added to your `package.json` file

```
"dependencies": {
  "bootstrap": "^5.3.8",
  "bootstrap-icons": "^1.13.1",
  …
}
```

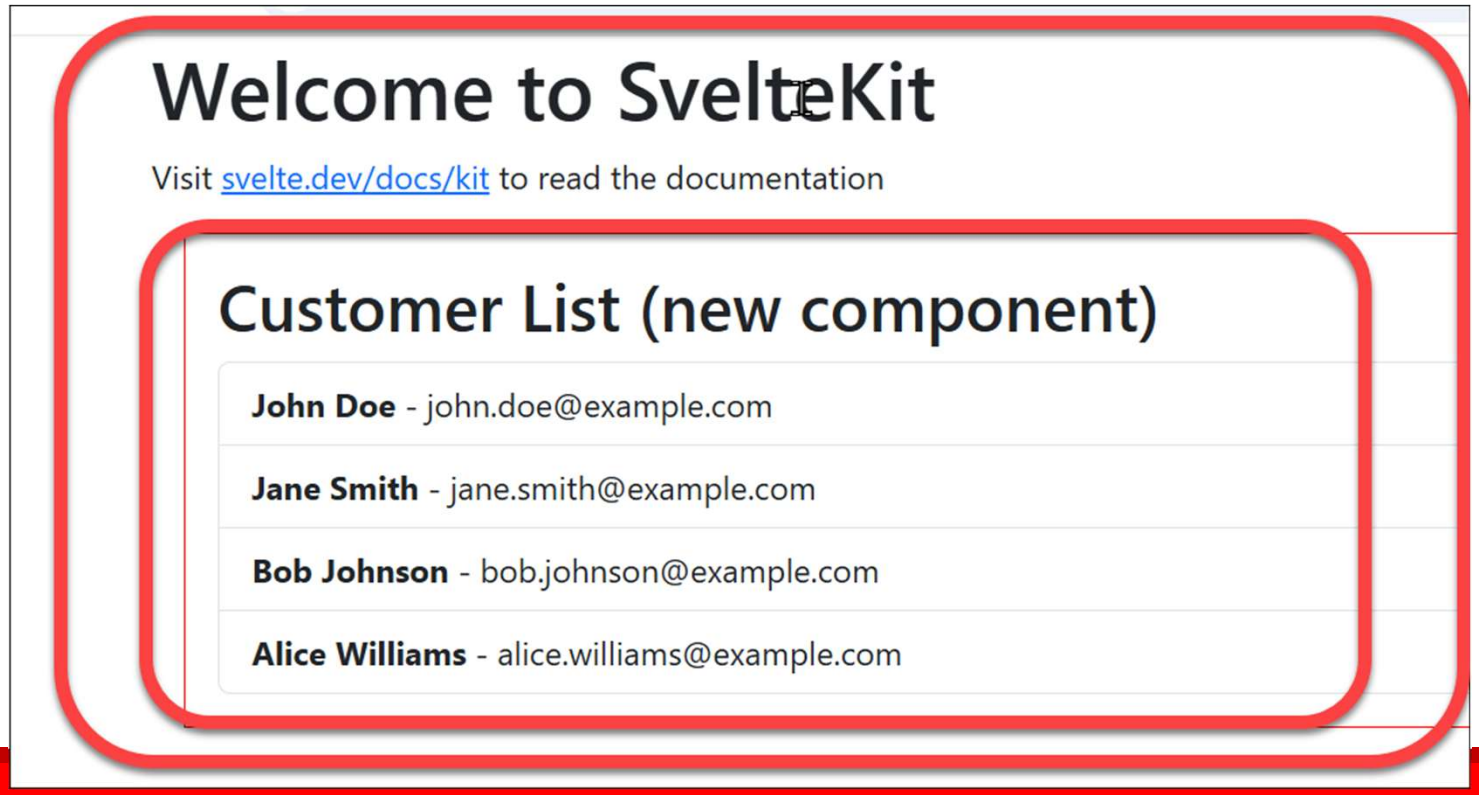
Use library in your app

- Two approaches:
  - Import directly in `+layout.svelte`
  - Import in a `shared.(s)css` file and import that one in the layout file
  - Both work fine. If you have a lot of overrides/manual css additions, the 2nd approach might work better (less cluttered layout file)

```
<!-- +layout.svelte -->
<script lang="ts">
  // using global css
  import 'bootstrap/dist/css/bootstrap.min.css';

  let { children } = $props();
</script>
```

Using the library

- You can now use CSS classes in files that are children of `+layout.svelte` (=basically all files)



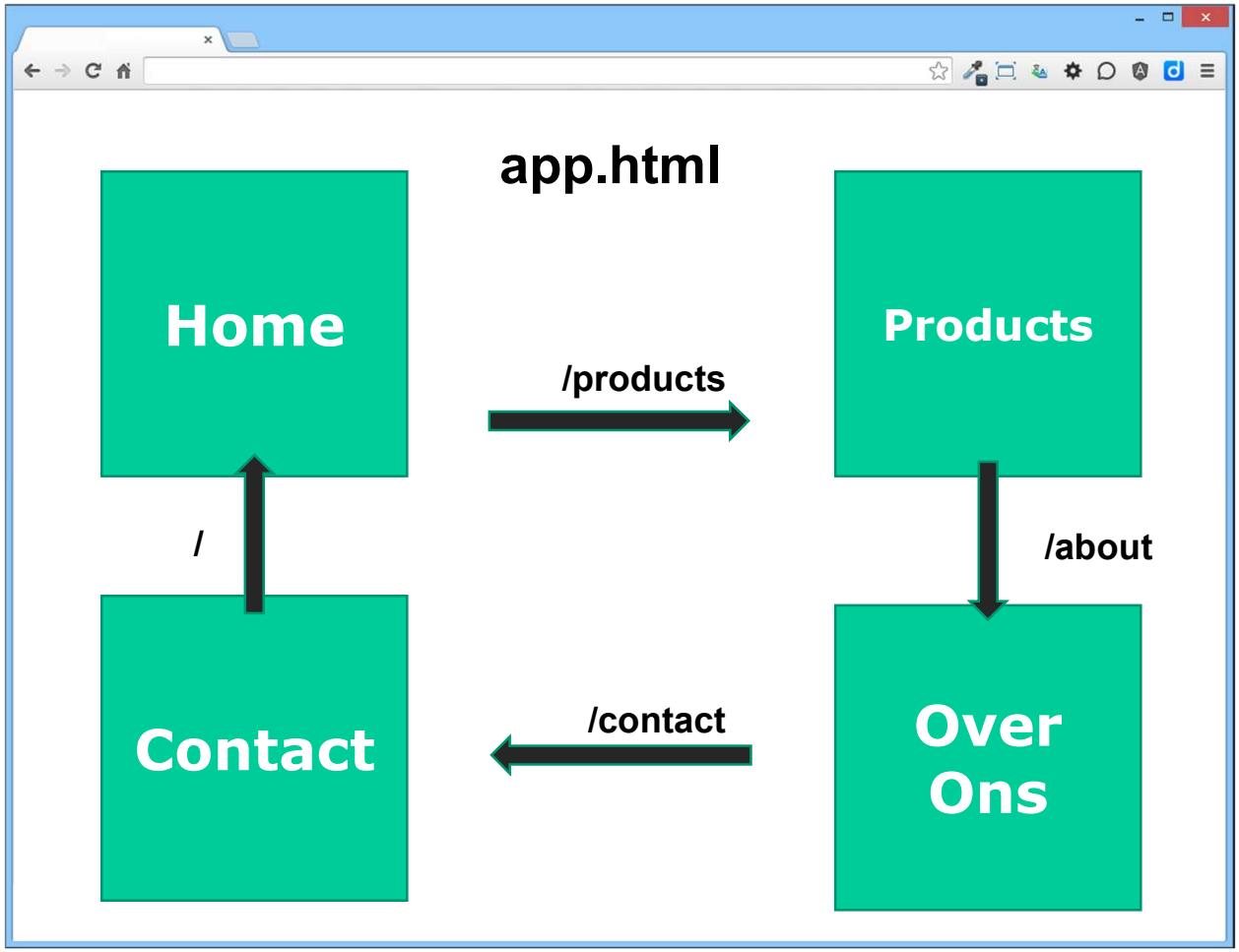
File Based routing

Core concept of SvelteKit – no need to configure the router separately

Routing topics

- `+page.svelte`
- `+layout.svelte`
- Nested layouts
- Router parameters `[id]`
- Dynamic routes

Routing architecture and goal



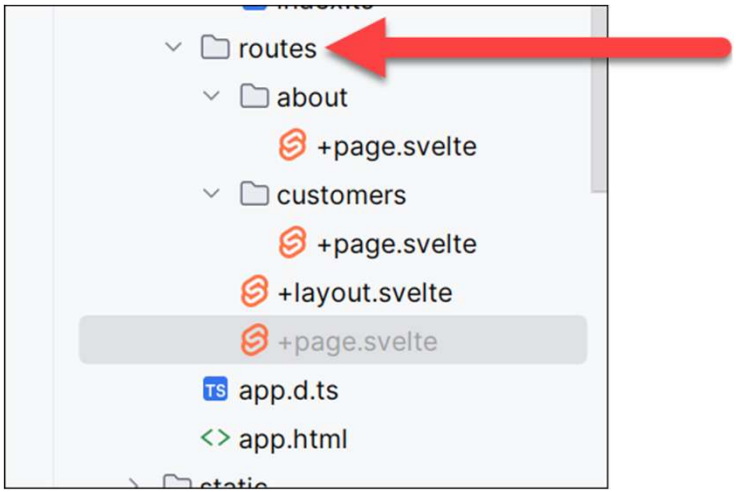
- Make use of SPA principle
- Making deep links possible

File based routing

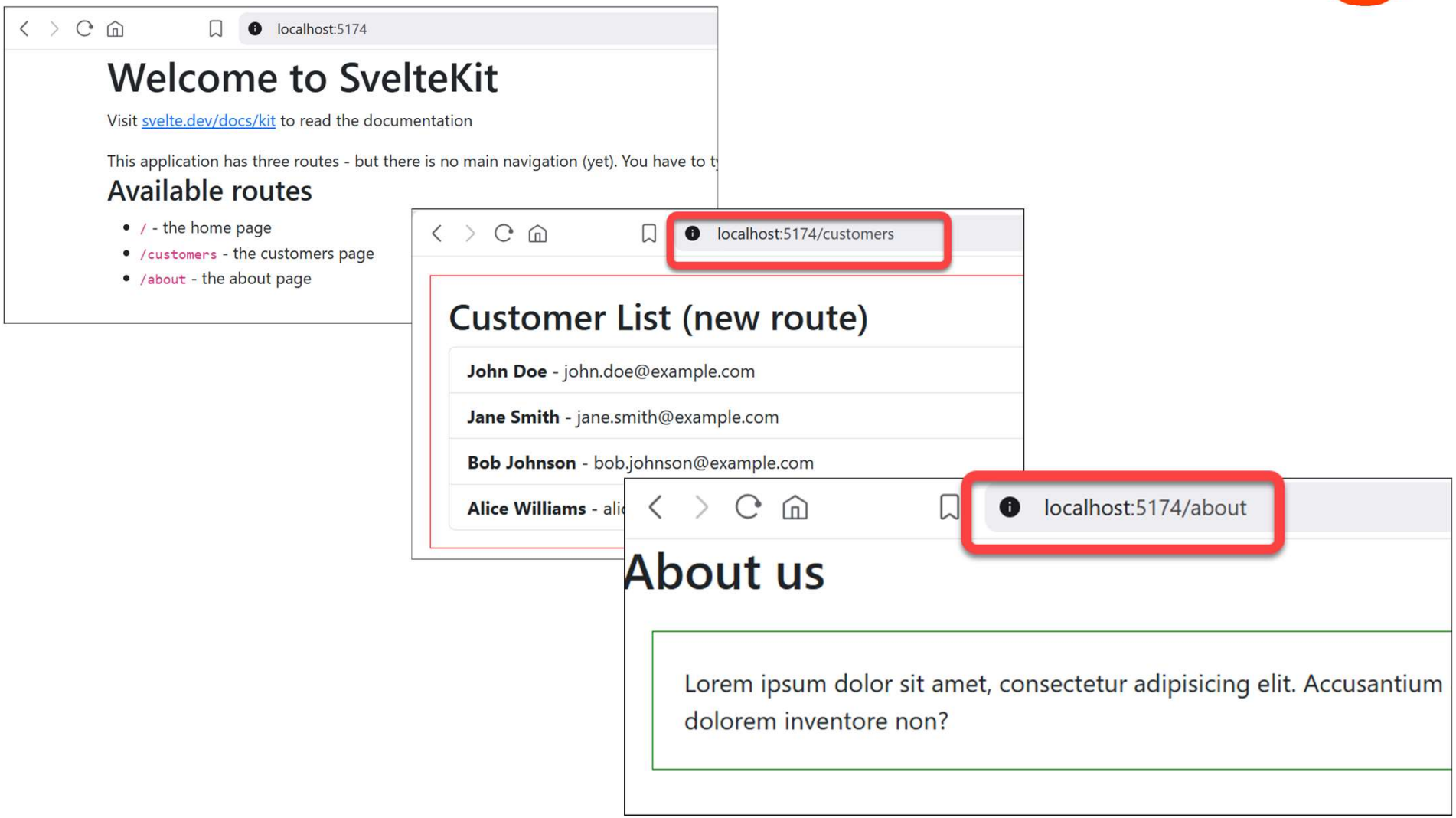
- The routes of your app (the URL paths that users can access) are defined by directories in your codebase:
  - `src/routes` is the root route
  - `src/routes/about` creates an /about route
  - `src/routes/blog/[slug]` creates a route with a parameter, `slug`,
  - It can be used to load data dynamically when a user requests a page like `/blog/hello-world`

Creating pages

- Create directories under `src/routes` with a `+page.svelte` component
  - The directory name is the name of the route
  - `+page.svelte` is the 'homepage' of that directory/route
  - Yes, there can be *a lot* of `+page.svelte` files in your project

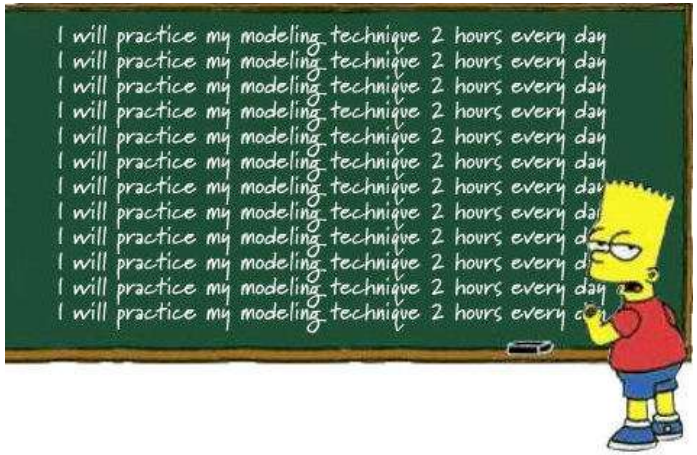


Three routes, no main navigation (yet)



Mini Workshop

- Use your previous application
- Create three routes. Create a component for each route
  - Tip: create a `+page.svelte` in each directory under `src/routes`
- Make sure you can navigate to the routes using the address bar in the browser



Main navigation

Creating top level navigation for your apps

Main navigation

- You don’t want to repeat navigation in every `+page.svelte` file
- Solution: create one navigation system and place it in `+layout.svelte` file
- Of course, you can also create a `<MyNavigation />` component and include it there
  - This keeps the `+layout.svelte` file tidy and less cluttered.

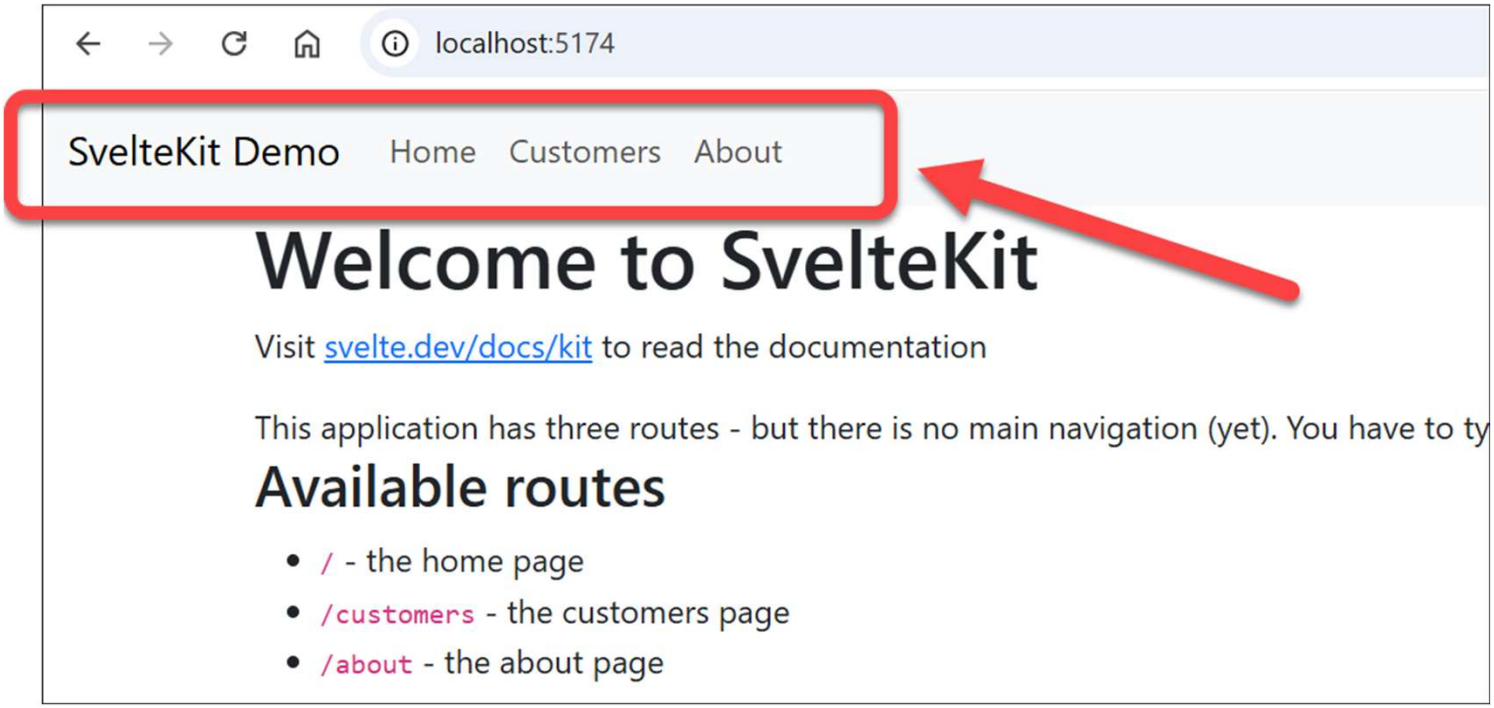
Adding the navigation bar

```
<!--+layout.svelte-->
<script lang="ts">
  …
  let { children } = $props();
</script>
…

<Navbar />

<main>
  <!-- Components will be rendered here-->
  {@render children()}
</main>
```

```
<!--src/lib/Navbar.svelte-->
<nav class="navbar navbar-expand-lg navbar-light bg-light">
   …
    <a class="nav-link" href="/">Home</a>
    …
</nav>
```

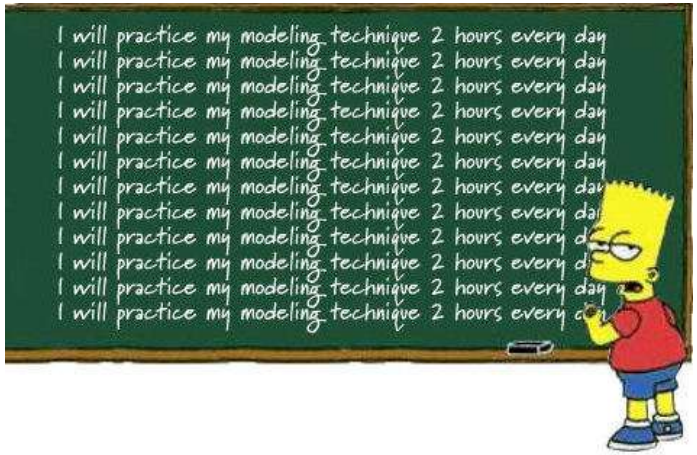
Navigating to routes

- Use a simple `<a href="…">` link.
  - No special `RouterLink` elements like in React, Vue or Angular



Mini Workshop

- Use your previous application
- Create main navigation for your app
  - Tip: place inside `+layout.svelte` at the root level
- Make sure you can navigate to the routes using the main navigation in the browser
- Example: `../420-sveltekit-navigation`



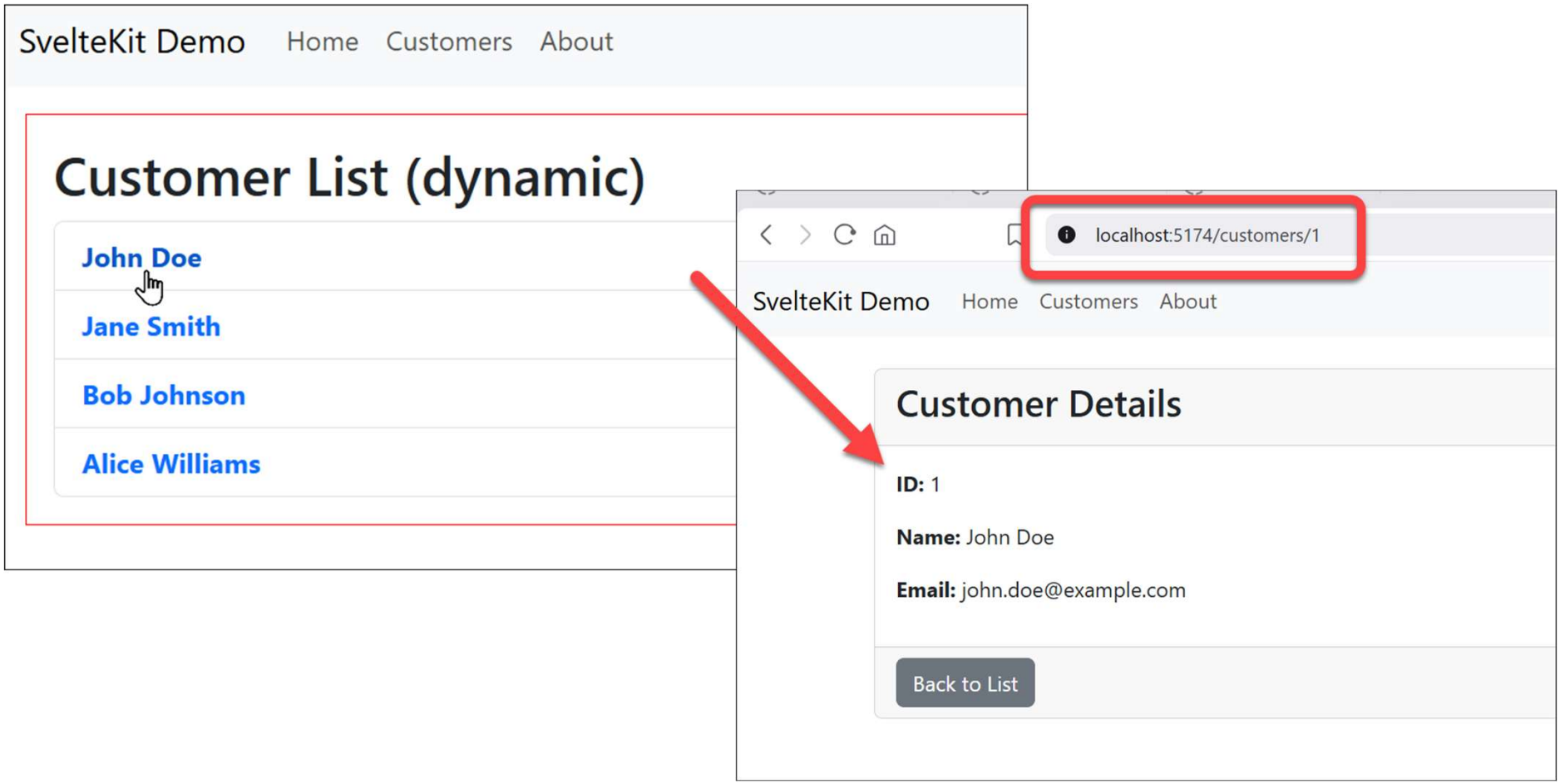
Route parameters

Creating dynamic routes, using a single page

Dynamic routes

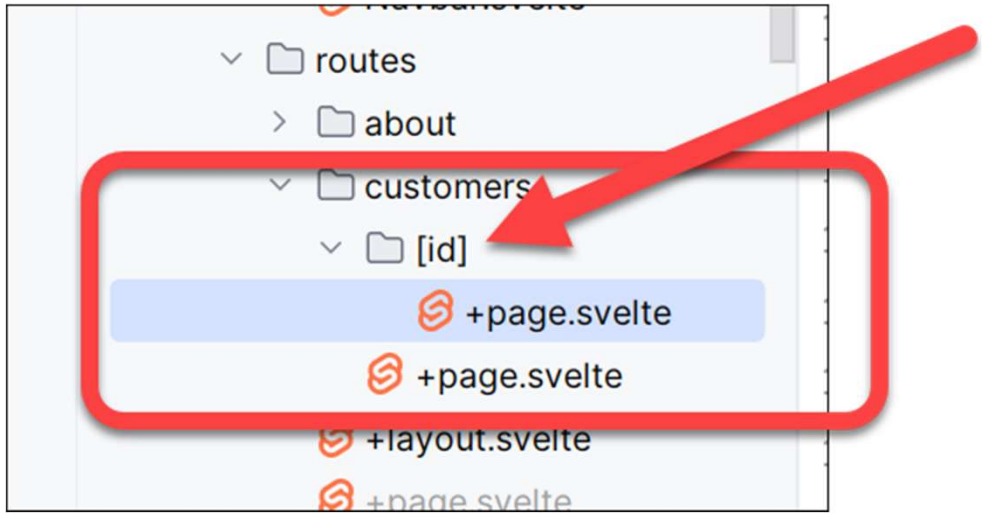
- A directory name in brackets `[...]` indicates a dynamic route.
  - It is filled in at runtime
  - For instance: `/src/routes/customers/[id]/+page.svelte`
- Multiple steps:
  - 1. Create dynamic folder
  - 2. Create dynamic link, sending the parameter(s)
  - 3. Fetch parameters from the route, load + show correct data

Goal:


SvelteKit Demo
Home
Customers
About
Customer List (dynamic)
John Doe
Jane Smith
Bob Johnson
Alice Williams
localhost:5174/customers/1
SvelteKit Demo
Home
Customers
About
Customer Details
ID: 1
Name: John Doe
Email: john.doe@example.com
Back to List

1. Create dynamic folder

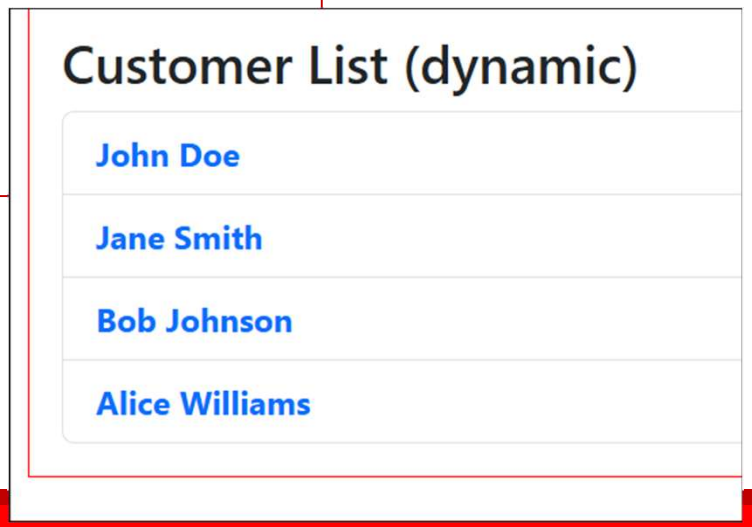
- Create a folder `[id]` and add `+page.svelte` to it
- `+page.svelte` is empty for now but will be filled later on to fetch details!



2. Create dynamic link

- In our `CustomerList`, we want to send the detail
  - In this case, we send an `id`.
  - Use curly brackets `{…}` inside an `#each`-loop for dynamic data

```
{#each customers as customer}
  <li class="list-group-item">
   <a href="/customers/{customer.id}">
    <strong>{customer.name}</strong>
   </a>
  </li>
{/each}
```



3. Fetch parameters from route, load data

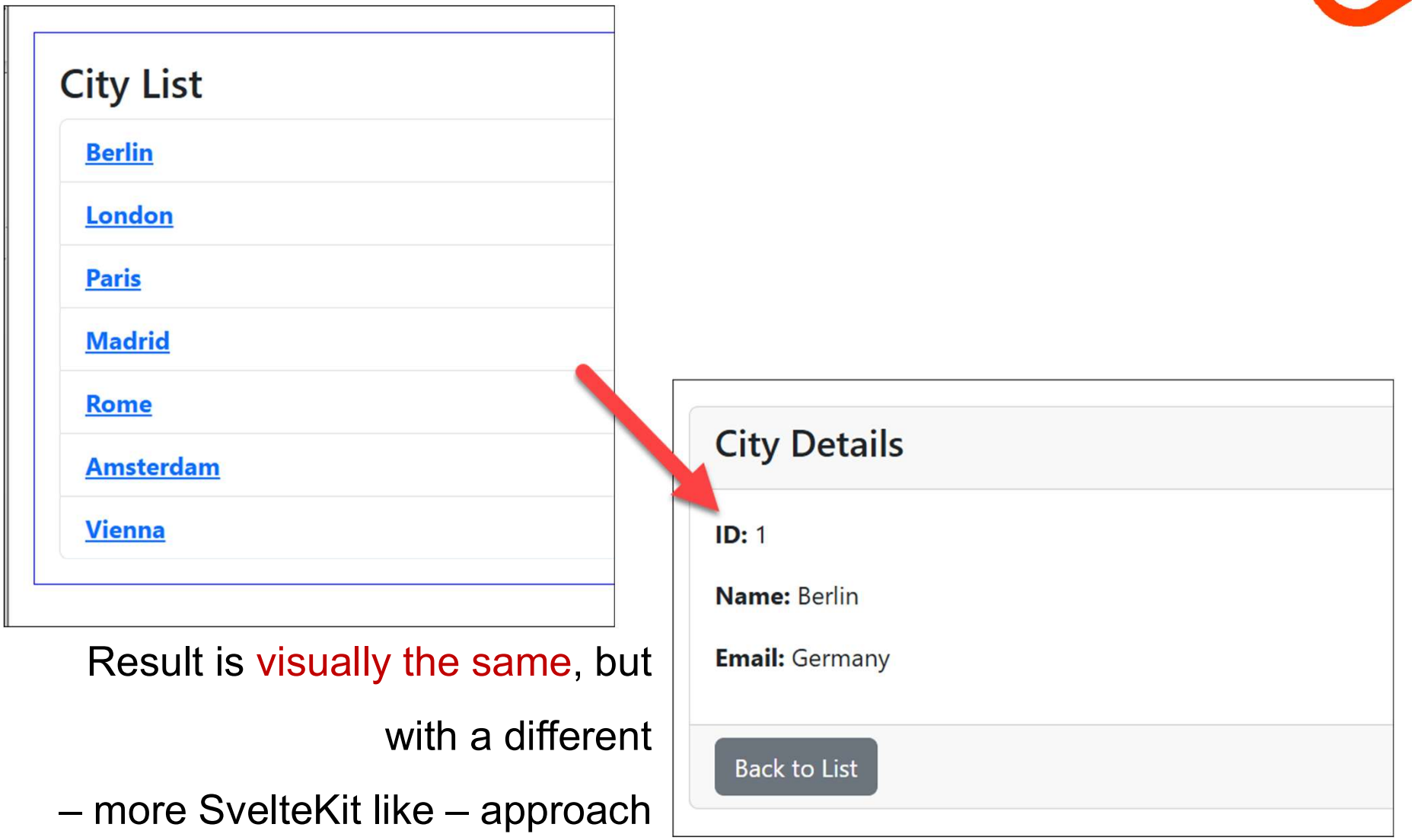
- This is the 'heavy lifting'. There are multiple approaches:
  - Using the `page.params.id` element from Svelte
  - Using the `load()` function.
- When using the first approach:

```
<script lang="ts">
  // Import the page store from SvelteKit to access route parameters
  import { page } from '$app/state';

  // Create a reactive variable for the customer ID
  const id = $derived(Number(page.params.id));

  // In real life, we would do a database lookup to fetch details.
  const customer = $derived(customers.find((c) => c.id === id));
</script>
```

3a – Alternative, using `+page.ts` and `load()`

- An alternative approach is to use:
  - 1. read `parameters.id` in a new `+page.ts`
  - 2. return the matching customer as `data` to the component
  - 3. use the data in `+page.svelte`
- The `+page.ts` runs *on the server*, before the page/UI is loaded
- This way, the component can focus on simply rendering the data instead of also fetching it.
  - This is a bit cleaner approach – preferred in larger applications.

Goal



Result is visually the same, but with a different – more SvelteKit like – approach

https://svelte.dev/docs/kit/load

svelte.dev/docs/kit/load

SVELTE Docs Tutorial Packages Playground Blog search

SVELTEKIT • CORE CONCEPTS

Loading data

Before a `+page.svelte` component (and its containing `+layout.svelte` components) can be rendered, we often need to get some data. This is done by defining `load` functions.

Page data

A `+page.svelte` file can have a sibling `+page.js` that exports a `load` function, the return value of which is available to the page via the `data` prop:

src/routes/blog/[slug]/+page.ts JS TS

```
import type { PageLoad } from './$types';

export const load: PageLoad = ({ params }) => {
  return {
    post: {
      title: `Title for ${params.slug} goes here`,
      content: `Content for ${params.slug} goes here`
    }
  };
};
```


Getting started
Introduction
Creating a project
Project types
Project structure
Web standards

Core concepts
Routing
Loading data
Form actions
Page options
State management
Remote functions

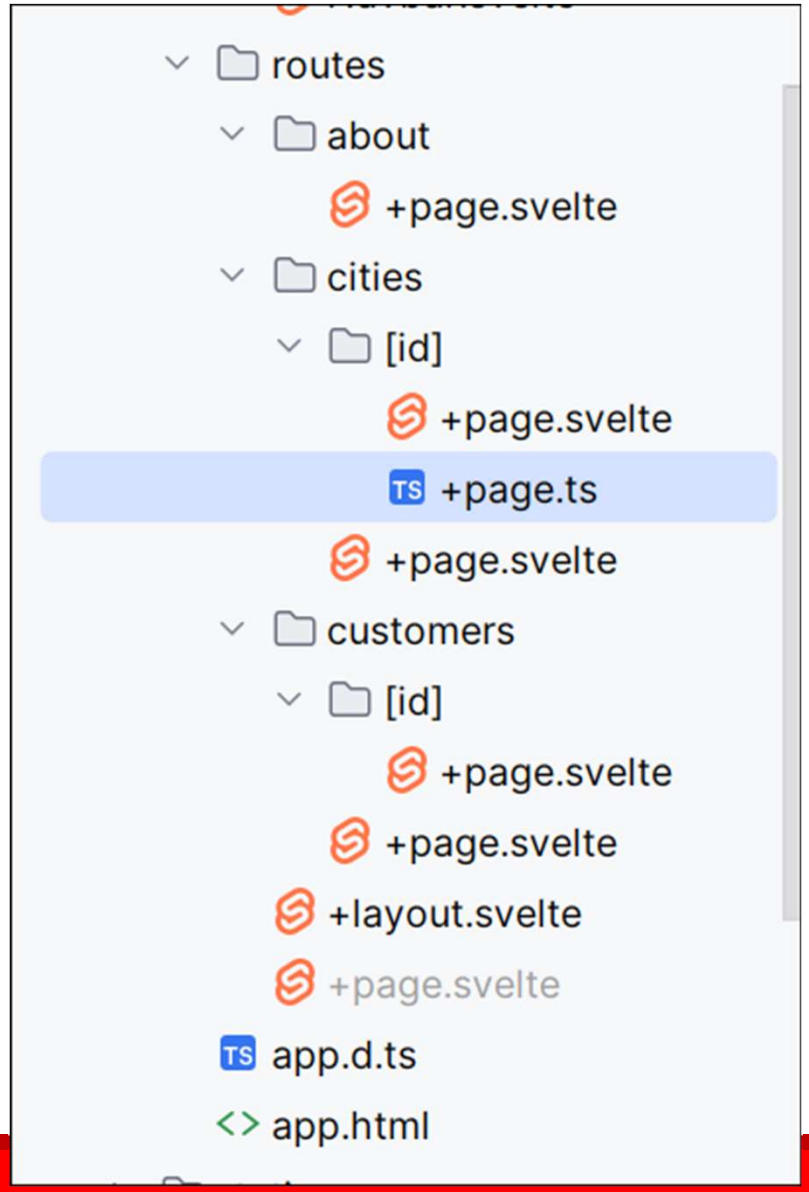
Build and deploy
Building your app
Adapters
Zero-config deployments
Node servers
Static site generation
Single-page apps
Cloudflare

We use Typescript, so `+page.ts`

```
import cities from '$lib/cities.json';
import type { PageLoad } from './$types';

// Export the load function that runs on the server
export const load: PageLoad = ({ params }) => {
  const id = Number(params.id);
  const city = cities.find((c) => c.id === id);
  // Return an object
  return {
   id,
   city
  };
};
```

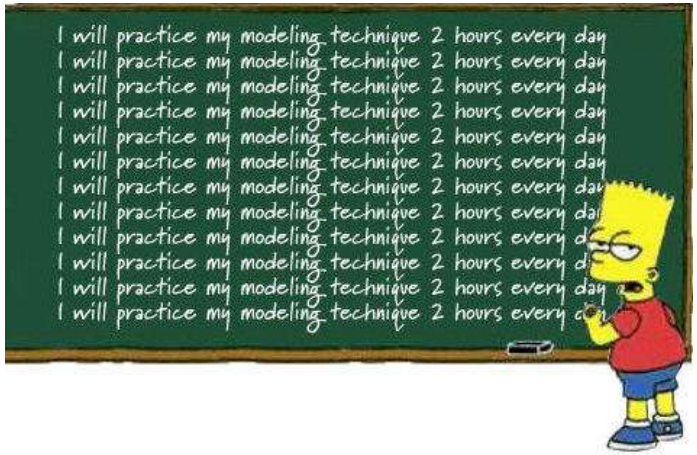
```
<!--+page.svelte for the cities/[id] route-->
<script lang="ts">
  import type { PageData } from './$types';
  // access page data loaded from +page.ts
  let { data }: { data: PageData } = $props();
  const city = $derived(data.city);
</script>
```

Project structure can get complex...



Workshop

- Use your previous application
- Create parent/child navigation and dynamic parameters
- Pick one of the approaches available for reading route parameters
- Example: `../430-sveltekit-details`



Nested navigation

Creating nested routes, using multiple `+layout.svelte` pages

Multiple `+layout.svelte` pages

- We can have multiple, nested `+layout.svelte` pages.
- Content of nested `+layout.svelte` pages is *added* to the main `+layout.svelte` page.
- Thus, we can create nested navigation

Layouts can be *nested*. Suppose we don't just have a single `/settings` page, but instead have nested pages like `/settings/profile` and `/settings/notifications` with a shared submenu (for a real-life example, see github.com/settings).

We can create a layout that only applies to pages below `/settings` (while inheriting the root layout with the top-level nav):

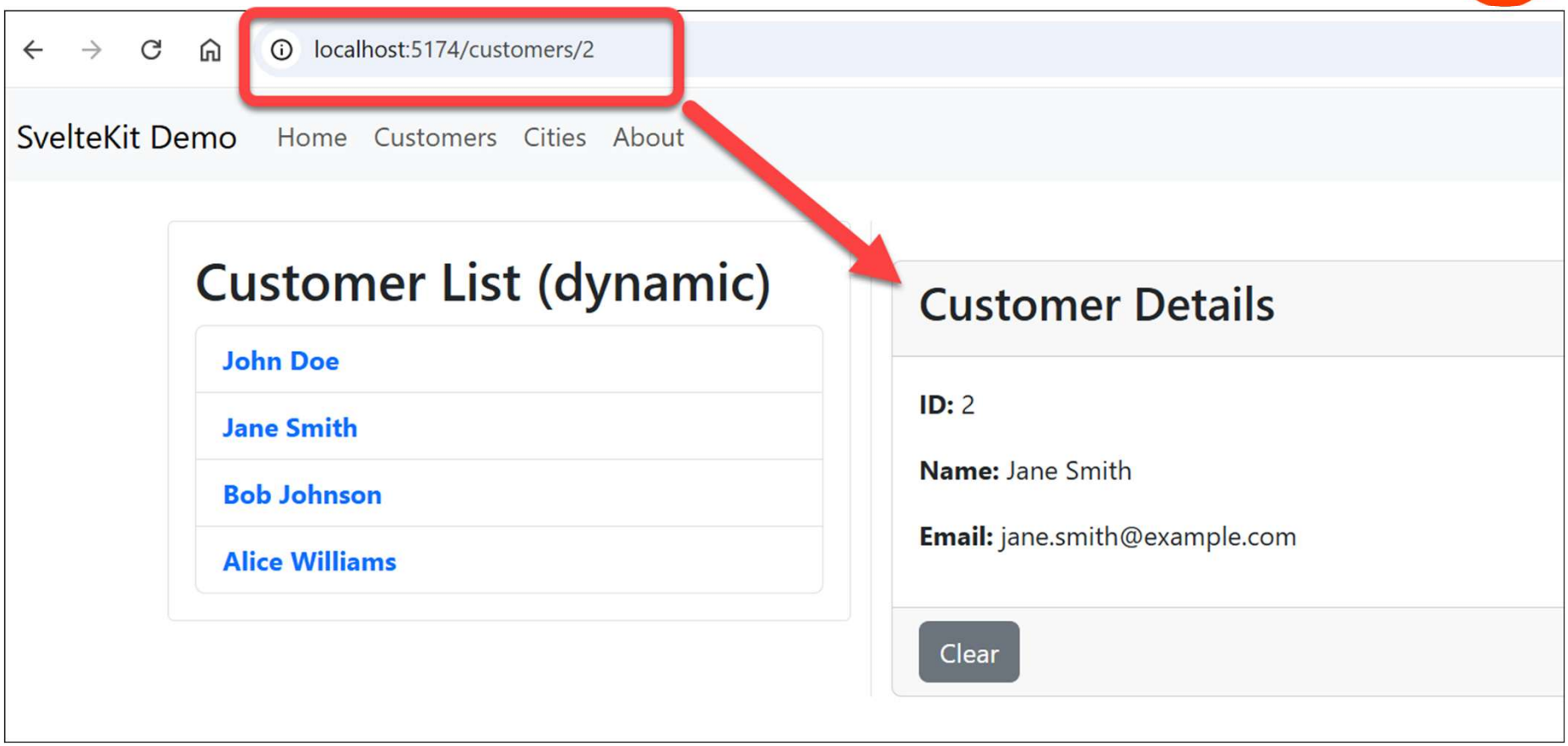
src/routes/settings/+layout.svelte JS TS

```
<script lang="ts">
  import type { LayoutProps } from './$types';

  let { data, children }: LayoutProps = $props();
</script>
```

Simple example

- Showing customer data in the same page, while still updating the browser address bar

Result



../440-sveltekit-nested-routing

Bottomline: `+layout.svelte` can be stacked

svelte.dev/docs/kit/advanced-routing#Advanced-layouts

SVELTE Docs Tutorial Packages Playground Blog search

Getting started
Introduction
Creating a project
Project types
Project structure
Web standards

Core concepts
Routing
Loading data
Form actions
Page options
State management
Remote functions

Build and deploy
Building your app
Adapters
Zero-config deployments
Node servers
Static site generation
Single-page apps
Cloudflare
Cloudflare Workers
Netlify
Vercel

Advanced layouts

By default, the *layout hierarchy* mirrors the *route hierarchy*. In some cases, that might not be what you want.

(group)

Perhaps you have some routes that are 'app' routes that should have one layout (e.g. `/dashboard` or `/item`), and others that are 'marketing' routes that should have a different layout (`/about` or `/testimonials`). We can group these routes with a directory whose name is wrapped in parentheses — unlike normal directories, `(app)` and `(marketing)` do not affect the URL pathname of the routes inside them:

```
src/routes/
│ (app)/
│ ├ dashboard/
│ ├ item/
│ └ +layout.svelte
│ (marketing)/
│ ├ about/
│ ├ testimonials/
│ └ +layout.svelte
├ admin/
└ +layout.svelte
```

You can also put a `+page` directly inside a `(group)`, for example if `/` should be an `(app)` or a `(marketing)` page.

https://svelte.dev/docs/kit/advanced-routing#Advanced-layouts

SvelteKit Next Steps

- advanced authentication
- Streaming
- advanced adapters
- advanced SSR
- Caching
- edge runtimes
- remote functions